

1 — Technical Architecture

What this document is: every vendor, service, and module in the Health Transformation platform, what each one actually does, and how they connect.

Verified against the codebase on 2026-09-07. Where this document and any older documentation disagree, this one is correct — it was written by reading the code, not the previous docs.

1.1 The system in one paragraph

The platform is **one Next.js web application** (hosted on Vercel) serving **four domains from a single deployment**, backed by **one Postgres database** (Supabase), **one headless CMS** (Sanity), and **one Python AI service** (FastAPI on Fly.io). A book — *Transforming Healthcare* — is the intellectual source material; it lives in this repo as a Markdown manuscript with its own build pipeline that produces the styled Word/PDF editions, and its framework is wired into the site's navigation and content taxonomy.

1.2 The three deployable pieces

Piece	What it is	Where it runs	Deploys when
Frontend	Next.js 16 app — the entire website, 189 pages, ~40 API routes	Vercel (region <code>iad1</code>)	Auto, on push to <code>main</code>
Backend	FastAPI service "HTR AI Brain" v4.2.0 — the AI Analyst	Fly.io (app <code>vhp-backend</code> , region <code>sjc</code>)	GitHub Action, only when <code>backend/**</code> changes
Book	Markdown manuscript + Python build pipeline → styled <code>.docx</code>	Local only (not deployed)	Manually, via <code>./book.sh</code>

The frontend is the only thing users hit directly. The backend is reached **only** through the frontend, which proxies to it (see §1.6).

1.3 Vendor-by-vendor: what each one is for

Vercel — frontend hosting

Hosts the Next.js app. Auto-deploys from `main`. Also runs the two scheduled jobs (`vercel.json`):

Cron	Schedule	Purpose
<code>/api/cron/revalidate</code>	<code>0 0 * * *</code> (daily, midnight)	Refresh cached content
<code>/api/cron/digest</code>	<code>0 14 * * 2</code> (Tuesdays, 14:00)	Send the email digest

Supabase — database, auth, and vector store

Does **three separate jobs**, which is easy to miss:

1. **Postgres database** — ~57 tables across 36 migrations (users, courses, progress, hospitals, state metrics, subscriptions, comments, API keys...).
2. **Authentication** — user accounts, sessions, password reset.
3. **Vector store for the AI** — the `pgvector` extension holds embeddings of all site content; the `hybrid_search_rag` Postgres function does the semantic + keyword search the AI Analyst runs on every question.

Row-Level Security (RLS) policies are defined in migrations `006` and `008`.

Sanity — content management (CMS)

Where editorial content is written and stored. **24 content types**, the most important being:

Type	What it holds
<code>academyModule</code>	Academy lesson bodies — 250 documents , the real course content
<code>policyAnalysis</code>	Analysis briefs
<code>caseStudy, webinar, report</code>	Editorial content
<code>post, dailyInsight, analystNote</code>	The Wire / news content
<code>definition</code>	Glossary terms
<code>hospital, rhtState, statePerformanceIndex</code>	Structured reference data
<code>author, instructor, category, ticker</code>	Supporting metadata

Editors work in **Sanity Studio**, embedded at `/studio` inside the app itself — not a separate site.

Fly.io — the Python AI backend

Runs the FastAPI "HTR AI Brain." This is the **only** reason Fly.io is in the stack. It powers the AI Analyst chat, the RAG pipeline, the Vermont live-data tools, and personalized-learning audio.

Configured (`backend/fly.toml`) as a single shared-CPU 1 GB machine, currently **scale-to-zero**: it sleeps when idle and wakes on the first request (5–15 s cold start). See doc 2 for the cost tradeoff.

The three AI vendors — and why there are three

This is the part most often misunderstood. **They do different jobs.**

Vendor	Model	Job	When it runs
Groq	<code>llama-3.1-8b-instant</code>	Writes chat answers for free/student users	Every chat message
Groq	<code>llama-3.3-70b-versatile</code>	Writes chat answers for subscribers	Every chat message
Anthropic	<code>claude-sonnet-4-6</code>	Writes chat answers for advisory/admin tier	Advisory-tier chats
OpenAI	<code>text-embedding-3-small</code>	Embeddings — turns content into searchable vectors	Content indexing + startup
OpenAI	TTS	Text-to-speech for personalized-learning audio	That feature only
OpenAI	<code>gpt-4o-mini</code>	Last-resort fallback if Groq is down	Rare

Practical consequences:

- **Groq writes most answers.** It is the primary chat brain, not OpenAI.
- **OpenAI is mostly invisible infrastructure** (search indexing). If the OpenAI key runs out of credit, chat *still works* — but semantic search degrades, and you'll see `credit_balance_exhausted` in the Fly logs.
- Model routing lives in `backend/services/llm.py` → `get_llm_for_role()`. Each tier has a fallback chain, so one vendor failing degrades rather than breaks the feature.

Stripe — payments and subscription tiers

Plan	Price	Grants role
Student	\$19/mo · \$180/yr	<code>student</code>
Subscriber	\$29/mo · \$279/yr	<code>subscriber</code>
Professional	\$99/mo · \$959/yr	<code>professional</code>
Team	\$23/seat/mo · \$221/seat/yr (2–25 seats)	<code>subscriber</code> per seat

Price IDs come from environment variables, so test and production Stripe accounts can differ. Webhook at `/api/stripe/webhook` keeps `subscriptions` and `user_roles` in sync.

Supporting services

- **Sentry** — error monitoring (`@sentry/nextjs`).
- **Resend** — transactional email.
- **Loops** — welcome/marketing email sequences (`lib/loops.ts`).
- **GitHub Actions** — CI and backend deploy (§1.8).

1.4 Multi-brand: four domains, one deployment

`lib/brand.ts` resolves a brand from the incoming `Host` header:

Domains	Brand	Difference
healthtransformationsolutions.org / .com	<code>solutions</code>	Full functionality, including Advisory
healthtransformationreview.org / .com	<code>review</code>	Advisory Services section hidden

Roughly 99% of the app is identical between them — only the wordmark, display name, and whether Advisory appears. There is **one** codebase and **one** Vercel deployment; nothing is forked per domain.

1.5 Access control: roles and the beta gate

Six roles, strictly ordered (`lib/auth.ts`). Each level includes everything below it:

free < subscriber < student < professional < advisory < admin

- `roleAtLeast(userRole, required)` is the single gate function.
- `requireRole()` redirects to `/upgrade` when a user lacks access.
- A user's role is the **highest** row found in the `user_roles` table.

Separately, the whole site currently sits behind a **beta gate**: a code is checked against the `beta_access_codes` table and, if valid, sets an `htr_beta` cookie. Beta codes can be scoped to a domain. `ALLOW_AUTH_BYPASS` is used in CI/testing to skip this.

1.6 How a question to the AI Analyst actually flows

This is the most intricate path in the system:

User types in chat (/chat or the sidebar widget)



Next.js API route `/api/chat` [Vercel]

↓ proxies via `PYTHON_BACKEND_URL`



FastAPI `/api/chat` [Fly.io]



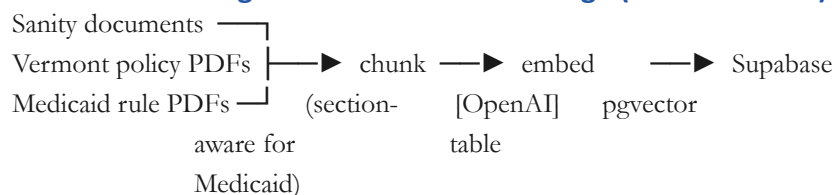
└─ 1. Auth + tier check → picks the model for the user's role

- └ 2. CLASSIFY the question (system prompt, routers/chat.py):
 - Vermont-operational? → call a live-data tool first
 - Vermont-policy? → retrieved docs + knowledge
 - National/general? → expert knowledge
- └ 3. RETRIEVE (services/retrieval.py):
 - embed the query [OpenAI embeddings]
 - hybrid_search_rag RPC, top_k=20 [Supabase pgvector]
 - boost Vermont-relevant results
 - re-rank to top 5 [FlashRank, optional]
- └ 4. GENERATE the answer [Groq · Anthropic · OpenAI fallback]
- └ 5. Attach citations, log the query (rag_query_log)

Live Vermont data tools the AI can call directly (`services/tools.py`) — these return real data, not retrieved text:

Tool	Returns
<code>query_vermont_system_summary()</code>	Vermont health-system overview
<code>query_vermont_hospital_financials(name)</code>	Operating margin, losses, 2028 projection
<code>query_vermont_bed_capacity(hospital, type)</code>	Live bed availability
<code>query_act167_recommendations(hospital)</code>	Act 167 (Oliver Wyman) recommendations
<code>find_best_transfer(from, acuity, specialty)</code>	Transfer routing
<code>query_vermont_hsa_population(hsa)</code>	Population by Health Service Area
<code>query_state_metrics(state)</code>	HTR performance index scores
<code>list_research_lab_tools(topic)</code>	Platform tool discovery

1.7 How content gets into the AI's knowledge (the RAG index)



- Triggered by `POST /api/ingest` on the Fly app (auth: `INGEST_SECRET`).

- Also triggered automatically by a **Sanity webhook** → `/api/ingest/webhook` (HMAC-verified), so publishing content re-indexes it.
- Medicaid eligibility PDFs are deliberately chunked at **rule/section boundaries** rather than by sentence window — splitting conditional eligibility logic across chunks produces wrong answers.
- `services/indexing.py` pulls 8 Sanity types: `policyAnalysis`, `post`, `academyModule`, `caseStudy`, `definition`, `analystNote`, `webinar`, `report`.

1.8 CI/CD

`ci.yml` runs on every push/PR to `main`:

Job	Steps
Frontend	<code>npm ci</code> → lint → <code>tsc --noEmit</code> → build → bundle-size budget
E2E	Build, start server, Playwright smoke tests (Chromium)
Backend	<code>pip install</code> → <code>ruff</code> lint → validate RAG golden dataset → mypy (non-blocking)

`fly-deploy.yml` deploys the backend to Fly.io — but **only when files under `backend/**` change**. Frontend deploys are handled by Vercel directly.

After a backend deploy, run `POST /api/ingest` on the Fly app to rebuild the RAG index. CI prints this reminder but does not do it automatically.

1.9 The Academy's dual-source content model

This trips people up constantly, so it is worth stating plainly.

Academy content lives in **two systems at once**:

System	Holds
Supabase	Course/track/lesson <i>structure</i> , ordering, publish flags, quizzes, and all <i>learner progress</i>
Sanity	The actual rich <i>lesson body</i> text

They are joined by one column: `lessons.sanity_slug`.

- If `sanity_slug` is `set` → the app renders the rich Sanity document.
- If `sanity_slug` is `null` → the app falls back to thin legacy `content_blocks` stored in the Supabase row.

So writing a beautiful lesson in Sanity does nothing until `sanity_slug` is set on the matching Supabase row. `scripts/link-sanity-slugs.mjs` does that linking; `scripts/audit-courses.mjs` reports which lessons are properly linked and rich.

Course structure is seeded from JSON files in `frontend/content/` (`course_seed.json`, `courses_tier1/2/3.json`) via `scripts/seed-all-courses.mjs` — **upsert-only, never deletes**.

1.10 The book pipeline

The book is both a product and the source of the site's intellectual framework.

File	Role
<code>HTR_Book_v42.md</code>	The build input. Markdown manuscript
<code>HTR_Book_v42.docx</code>	Generated output — never hand-edit
<code>HTR_Book_v42.pdf</code>	Exported by the author from Google Docs
<code>book-build/build_docx.py</code>	The engine: pandoc + a Python post-pass that renders callouts, stat strips, tables, TOC, cover
<code>book-build/make_reference.py</code>	Builds <code>reference.docx</code> , the Word stylesheet
<code>book-build/check_manuscript.py</code>	Structural validator — run before building
<code>book-build/sync_from_gdocs.py</code>	Finds the author's Google Docs edits to fold back into the Markdown
<code>./book.sh</code>	Wrapper: <code>who</code> / <code>mine</code> / <code>claud</code> (handoff lock), <code>check</code> , <code>build</code>

The framework in the book drives the site's taxonomy. As of v46: five pillars (Policy → Technology → Economics → Clinical → Operations) held to a cross-cutting **Equity Imperative**. `frontend/lib/taxonomy/pillars.ts` is the single source of truth for that structure in code.

1.11 Key frontend modules

Path	Responsibility
<code>lib/taxonomy/</code>	Source of truth for pillars, chapters, tools, programs
<code>lib/auth.ts</code>	Roles, <code>roleAtLeast</code> , <code>requireRole</code>
<code>lib/brand.ts</code> / <code>brand-server.ts</code>	Multi-domain brand resolution

Path	Responsibility
<code>lib/db/</code>	Supabase query layer (academy, hospitals, states, time-series)
<code>lib/sanity.ts, sanity-fetch.ts</code>	Sanity client + queries
<code>lib/stripe.ts</code>	Plan definitions and checkout
<code>lib/data/</code>	Static/reference datasets (HTI time series, RHT program, state comparison)
<code>lib/narration.ts</code>	Book audio narration tracks
<code>lib/rate-limit.ts</code>	API rate limiting
<code>app/studio/</code>	Embedded Sanity Studio

1.12 Dependency map (what breaks what)

If this goes down	What breaks	What still works
Vercel	Everything user-facing	—
Supabase	Login, Academy, dashboards, AI retrieval	Static marketing pages
Sanity	New content, lesson bodies, Studio	Cached pages, Academy structure, AI (index already built)
Fly.io backend	AI Analyst chat only	Entire rest of the site
Groq	Chat degrades to fallback models	Everything else
OpenAI	Semantic search quality, TTS audio	Chat still answers (Groq)
Stripe	New signups/upgrades	Existing users keep their access